

Lecture 5:

Multi-Byte Integers

CSE 29: Systems Programming and Software Tools

Aaron Schulman (Shalev)



Today's Lecture

- We need to mask (and shift) bits to do code point analysis
- **Review: We need large size integers to store large code points (int32_t)**
- **We may run into issues with too small integers making results look strange.**

Data types in typically used in C

Integer data types

- **char/int8_t** = 'A' (1 byte -8 bits– max 127) - *Signed*
 - **int/int32_t** = 42 (4 bytes -32 bits– max 2 billion) - *Signed*
 - **unsigned char/uint8_t** = (1 byte –8 bits- max 255) - *Unsigned*
 - **unsigned int/uint32_t** = (4 bytes -32 bits- max 4 billion)
- Precisely named integers (e.g., int8_t) are in <stdint.h>

Refresher – Integers in Computers

Integers: 64 bits (8 bytes) – “Word” Size

00000000	00000000	00000000	00000000	00000000	00000000	00000000	00000000
1 byte	1 byte	1 byte	1 byte	1 byte	1 byte	1 byte	1 byte

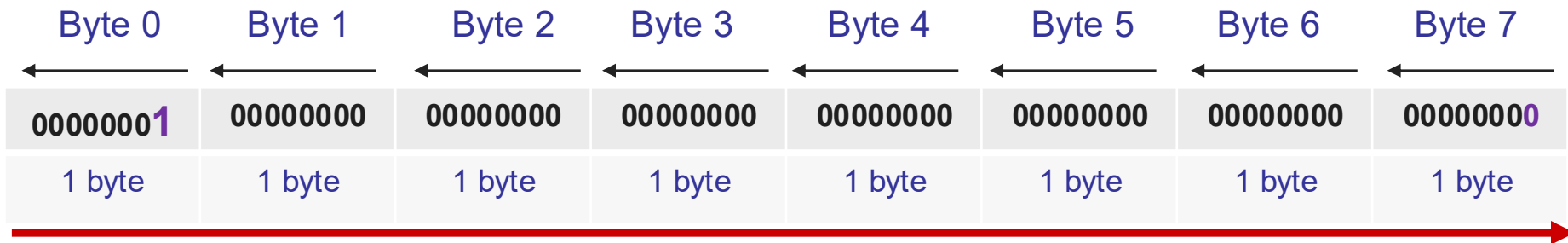
1 integer = 64 bits (~18 quintillion)

1 address = 64 bits (17,179,869,184 gigabytes)

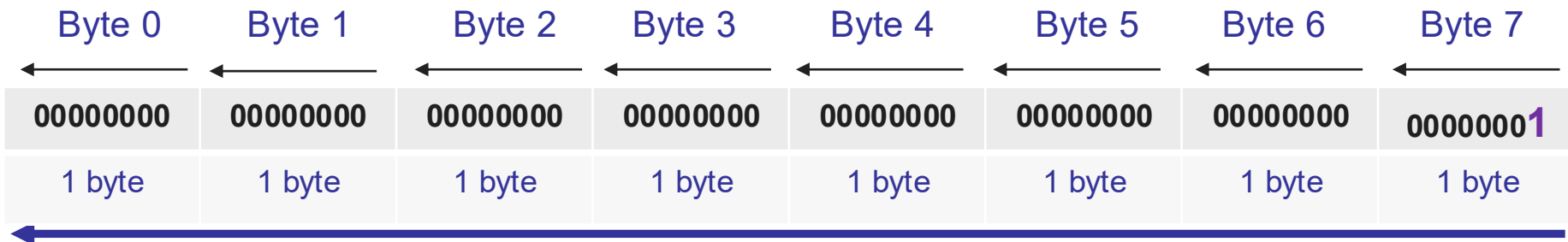
Older computers used 32-bit words:

$$2^{32} = 1024 * 1024 * 1024 * 4 = 4 \text{ gigabytes}$$

What is 1 as 64-bit number in memory?



Little Endian



Big Endian

Hex helps humans read multi-byte ints

Hexadecimal notation divides binary data into 4-bit groups:

111111000101101101111111001101101110100010111100101111010000101
0xFC5B7F9B745E5E85

1111	1100	0101	1011	0111	1111	1001	1011	0111	0100	0101	1110	0101	1110	1000	0101
F	C	5	B	7	F	9	B	7	4	5	E	5	E	8	5

Lecture Review

1. What happens if you store a multi-byte integer in a single-byte integer?
2. Is a 32-bit integer enough to store a number in the trillions?
3. If I have an integer made of 11 bytes, what integer types can I store it in?